

ATIVIDADE - ENCONTRO 10

Programação Paralela | 2026.1

Aluno: Pedro Pinheiro

Professor: Philippe Leal

Data: 16 de julho de 2026

1. Deadlock e starvation

a) Deadlock

Deadlock, ou impasse, é uma situação em que duas ou mais threads ficam bloqueadas de forma permanente porque cada uma espera por um recurso que está sendo mantido por outra. Nenhuma delas consegue avançar e, como consequência, também não libera o recurso que já possui. Um exemplo clássico ocorre quando a thread A bloqueia o recurso 1 e tenta obter o recurso 2, enquanto a thread B já bloqueou o recurso 2 e tenta obter o recurso 1. Forma-se, assim, uma espera circular.

Em sistemas concorrentes, o deadlock costuma ser associado a quatro condições que, quando existem simultaneamente, tornam o impasse possível: exclusão mútua, pois determinado recurso só pode ser usado por uma thread de cada vez; posse e espera, quando uma thread mantém um recurso enquanto aguarda outro; não preempção, porque o recurso não é retirado à força de quem o possui; e espera circular, quando existe uma cadeia fechada de threads esperando umas pelas outras.

Um deadlock pode paralisar somente as threads envolvidas ou comprometer uma parte maior do programa. Ele também pode ser difícil de reproduzir, pois depende da ordem de execução e do instante em que cada thread solicita os bloqueios. Entre as medidas de prevenção estão adquirir locks sempre na mesma ordem, evitar manter um lock enquanto se solicita outro, reduzir o tempo das regiões críticas, usar tentativas com limite de tempo e projetar estruturas que diminuam o compartilhamento de estado.

É importante distinguir deadlock de uma simples espera. Em uma espera normal, existe a expectativa de que alguma condição mude e a thread continue. No deadlock, nenhuma das threads envolvidas consegue provocar a mudança necessária, de modo que o progresso não ocorre sem uma intervenção externa ou encerramento do processo. [1][2]

b) Starvation

Starvation, ou inanição, ocorre quando uma thread permanece pronta ou repetidamente tenta executar, mas não recebe tempo de processador ou acesso a um recurso por um período indefinido. Ao contrário do deadlock, o sistema como um todo pode continuar funcionando: outras threads executam e concluem suas tarefas, enquanto uma thread específica é continuamente desfavorecida.

Esse problema pode surgir quando há prioridades muito diferentes, quando o escalonador sempre favorece as mesmas threads, quando um lock não oferece política justa de atendimento

ou quando novas requisições conseguem passar à frente de uma requisição antiga. No problema dos leitores e escritores, por exemplo, uma política que sempre permita a entrada de novos leitores pode fazer um escritor esperar indefinidamente.

Uma forma de reduzir starvation é empregar filas de atendimento, políticas de justiça, envelhecimento de prioridade - aumento gradual da prioridade de quem espera há mais tempo - e limites para a quantidade de operações consecutivas de um mesmo grupo. Portanto, deadlock significa que um conjunto de threads não consegue avançar por causa de uma dependência circular; starvation significa que uma thread não avança porque outras recebem continuamente a oportunidade de usar os recursos. [2]

2. Problema do Produtor-Consumidor

O problema do Produtor-Consumidor é um modelo clássico de sincronização em que uma ou mais threads produtoras criam dados e os colocam em uma área compartilhada, enquanto uma ou mais threads consumidoras retiram esses dados para processá-los. Essa área normalmente é representada por uma fila ou buffer de tamanho limitado.

A dificuldade aparece porque produtores e consumidores executam concorrentemente. Se duas threads alterarem a fila ao mesmo tempo sem proteção, podem ocorrer condições de corrida, perda de dados ou inconsistência na estrutura. Além disso, um produtor não deve inserir um item quando o buffer está cheio, e um consumidor não deve retirar um item quando o buffer está vazio.

A solução precisa garantir três propriedades principais:

- Exclusão mútua: somente uma thread por vez modifica a estrutura interna do buffer.
- Espera por condição: produtores esperam quando o buffer está cheio e consumidores esperam quando ele está vazio.
- Notificação: após inserir ou retirar um item, a thread sinaliza que o estado do buffer mudou e que outra thread pode continuar.

Uma implementação comum utiliza um mutex ou monitor para proteger o buffer e duas variáveis de condição: uma representa a existência de espaço disponível e outra representa a existência de itens disponíveis. Quando uma thread chama uma operação de espera em uma variável de condição, ela libera temporariamente o lock, permitindo que outra thread entre na região crítica e altere o estado. Ao ser acordada, a thread recupera o lock antes de continuar.

A condição deve ser verificada em um laço, e não apenas com uma decisão simples, porque a thread pode acordar e descobrir que outra thread consumiu a oportunidade antes dela. Assim, o produtor espera enquanto a fila estiver cheia, e o consumidor espera enquanto estiver vazia. Essa organização evita espera ocupada, na qual uma thread ficaria gastando processamento ao verificar continuamente se a situação mudou. [3][4]

3. Código em Ruby utilizando Monitor

O código a seguir implementa o Produtor-Consumidor com um buffer limitado. A classe inclui `MonitorMixin`, fornecido pela biblioteca padrão do Ruby. Com isso, o método `synchronize` cria uma seção de exclusão mútua, e `new_cond` cria variáveis de condição associadas ao monitor. A documentação do Ruby define o monitor como uma estrutura cujos métodos protegidos executam com exclusão mútua, isto é, no máximo uma thread ocupa a região protegida em determinado instante. [4][5]

```

require 'monitor'

class BufferLimitado
  include MonitorMixin

  def initialize(capacidade)
    super()
    @capacidade = capacidade
    @fila = []

    @nao_vazio = new_cond
    @nao_cheio = new_cond
  end

  def produzir(item, produtor)
    synchronize do
      @nao_cheio.wait_while do
        @fila.length >= @capacidade
      end

      @fila << item
      puts "#{produtor} produziu #{item} " \
        "(buffer: #{@fila.inspect})"

      @nao_vazio.signal
    end
  end

  def consumir(consumidor)
    synchronize do
      @nao_vazio.wait_while do
        @fila.empty?
      end

      item = @fila.shift
      puts "#{consumidor} consumiu #{item} " \
        "(buffer: #{@fila.inspect})"

      @nao_cheio.signal
    end
  end
end

buffer = BufferLimitado.new(3)

produtor_1 = Thread.new do
  5.times do |i|
    sleep(rand * 0.3)
    buffer.produzir("P1-#{i + 1}", "Produtor 1")
  end
end

produtor_2 = Thread.new do
  5.times do |i|
    sleep(rand * 0.3)
    buffer.produzir("P2-#{i + 1}", "Produtor 2")
  end
end

consumidor_1 = Thread.new do
  5.times do
    sleep(rand * 0.4)
    buffer.consumir("Consumidor 1")
  end
end

consumidor_2 = Thread.new do
  5.times do
    sleep(rand * 0.4)
    buffer.consumir("Consumidor 2")
  end
end

```

```
[
  produtor_1,
  produtor_2,
  consumidor_1,
  consumidor_2
].each(&:join)

puts "Processamento concluído."
```

A inicialização chama `super()` porque `MonitorMixin` precisa preparar o monitor interno. As variáveis `@nao_vazio` e `@nao_cheio` são condições distintas: a primeira é usada por consumidores que aguardam a chegada de dados; a segunda, por produtores que aguardam espaço.

No método `produzir`, `synchronize` garante que a verificação do tamanho e a inserção ocorram atomicamente em relação às demais threads. `wait_while` libera o monitor enquanto a fila estiver cheia e volta a adquirir o lock quando a thread for acordada. Após inserir, `signal` acorda uma thread que esteja esperando na condição `@nao_vazio`.

No método `consumir` acontece o processo inverso. A thread espera enquanto a fila estiver vazia, remove o primeiro item e sinaliza `@nao_cheio`, permitindo que um produtor bloqueado tente inserir. Os `sleeps` não fazem parte da sincronização; foram incluídos apenas para simular tempos diferentes de produção e consumo e tornar a concorrência visível na saída.

O método `signal` acorda uma thread em espera. Caso fosse necessário acordar todas as threads esperando a mesma condição, poderia ser utilizado `broadcast`. A operação `wait` libera o lock do monitor e o `readquire` antes de retornar, o que permite que a alteração do estado compartilhado seja feita com segurança. [5]

4. Problema dos Leitores e Escritores

O Problema dos Leitores e Escritores descreve o acesso concorrente a uma estrutura compartilhada, como um arquivo, banco de dados, cache ou tabela em memória. As threads são divididas em leitores, que apenas consultam os dados, e escritores, que podem modificá-los.

Como a leitura não altera o conteúdo, vários leitores podem acessar simultaneamente sem causar inconsistência. Já um escritor precisa de acesso exclusivo: enquanto ele estiver escrevendo, nenhum outro escritor e nenhum leitor devem usar a estrutura. O objetivo da sincronização é permitir o máximo de paralelismo seguro sem violar a integridade dos dados.

Uma solução básica mantém um contador de leitores ativos. O primeiro leitor bloqueia o recurso contra escritores; leitores adicionais apenas incrementam o contador e podem ler em paralelo. Quando um leitor termina, decrementa o contador, e o último leitor libera o recurso para um escritor. Escritores, por sua vez, adquirem acesso exclusivo antes de modificar os dados.

Entretanto, a política de prioridade muda o comportamento do sistema. Em uma solução com preferência para leitores, novos leitores podem entrar mesmo quando existe um escritor esperando. Isso melhora a vazão das leituras, mas pode causar starvation do escritor. Em uma solução com preferência para escritores, a chegada de um escritor impede a entrada de novos leitores; porém, uma sequência contínua de escritores pode atrasar leitores.

Uma solução justa utiliza uma fila ou ordem de chegada, impedindo que requisições novas ultrapassem indefinidamente as antigas. Bibliotecas modernas costumam fornecer locks de leitura e escrita, também chamados read-write locks. Esses locks permitem múltiplos bloqueios de leitura ou um único bloqueio de escrita, mas nunca as duas categorias simultaneamente.

O problema mostra que sincronização não significa somente impedir acesso simultâneo. Também envolve estabelecer uma política de progresso e justiça. Uma implementação correta preserva a exclusão dos escritores, permite leituras paralelas quando seguro e evita que um tipo de participante espere indefinidamente. [3]

Referências

- [1] ORACLE. About Deadlocks. Oracle Developer Studio Thread Analyzer User's Guide. Disponível em: https://docs.oracle.com/cd/E60778_01/html/E60750/gkgml.html. Acesso em: 16 jul. 2026.
- [2] ORACLE. Liveness e Synchronization. The Java Tutorials - Concurrency. Disponível em: <https://docs.oracle.com/javase/tutorial/essential/concurrency/>. Acesso em: 16 jul. 2026.
- [3] MILLER, Barton P. CS 537 - Introduction to Operating Systems: Lecture Notes. University of Wisconsin-Madison. Seções sobre semáforos, Produtor-Consumidor e Leitores-Escritores. Disponível em: <https://pages.cs.wisc.edu/~bart/537/lecturenotes/>. Acesso em: 16 jul. 2026.
- [4] RUBY. MonitorMixin - Ruby Standard Library Documentation. Disponível em: <https://ruby-doc.org/3.3/exts/monitor/MonitorMixin.html>. Acesso em: 16 jul. 2026.
- [5] RUBY. Monitor e MonitorMixin::ConditionVariable - Ruby Standard Library Documentation. Disponível em: <https://ruby-doc.org/3.4.1/exts/monitor/Monitor.html> e <https://ruby-doc.org/stdlib-3.0.1/libdoc/monitor/rdoc/MonitorMixin/ConditionVariable.html>. Acesso em: 16 jul. 2026.